Neural Adversarial Training for Semi-supervised Japanese Predicate-argument Structure Analysis

Shuhei Kurita[$†‡] Daisuke Kawahara[$†] Sadao Kurohashi[$†‡]
[$]Graduate School of Informatics, Kyoto University
[†]CREST, JST [‡]ACT-I, JST
{kurita, dk, kuro}@nlp.ist.i.kyoto-u.ac.jp

Abstract

Japanese predicate-argument structure (PAS) analysis involves zero anaphora resolution, which is notoriously difficult. To improve the performance of Japanese PAS analysis, it is straightforward to increase the size of corpora annotated with PAS. However, since it is prohibitively expensive, it is promising to take advantage of a large amount of raw corpora. In this paper, we propose a novel Japanese PAS analysis model based on semi-supervised adversarial training with a raw corpus. In our experiments, our model outperforms existing state-of-the-art models for Japanese PAS analysis.

1 Introduction

In pro-drop languages, such as Japanese and Chinese, pronouns are frequently omitted when they are inferable from their contexts and background knowledge. The natural language processing (NLP) task for detecting such omitted pronouns and searching for their antecedents is called zero anaphora resolution. This task is essential for downstream NLP tasks, such as information extraction and summarization.

For Japanese, zero anaphora resolution is usually conducted within predicate-argument structure (PAS) analysis as a task of finding an omitted argument for a predicate. PAS analysis is a task to find an argument for each case of a predicate. For Japanese PAS analysis, the ga (nominative, NOM), wo (accusative, ACC) and ni (dative, DAT) cases are generally handled. To develop models for Japanese PAS analysis, supervised learning methods using annotated corpora have been applied on the basis of morpho-syntactic clues. However, omitted pronouns have few clues and thus these models try to learn relations between a predicate and its (omitted) argument from the annotated corpora. The annotated corpora consist of several tens of thousands sentences, and it is difficult to learn predicate-argument relations or selectional preferences from such small-scale corpora. The state-of-the-art models for Japanese PAS analysis achieve an accuracy of around 50% for zero pronouns (Ouchi et al., 2015; Shibata et al., 2016; Iida et al., 2016; Ouchi et al., 2017; Matsubayashi and Inui, 2017).

A promising way to solve this data scarcity problem is enhancing models with a large amount of raw corpora. There are two major approaches to using raw corpora: extracting knowledge from raw corpora beforehand (Sasano and Kurohashi, 2011; Shibata et al., 2016) and using raw corpora for data augmentation (Liu et al., 2017b).

In traditional studies on Japanese PAS analysis, selectional preferences are extracted from raw corpora beforehand and are used in PAS analysis models. For example, Sasano and Kurohashi (2011) propose a supervised model for Japanese PAS analysis based on case frames, which are automatically acquired from a raw corpus by clustering predicate-argument structures. However, case frames are not based on distributed representations of words and have a data sparseness problem even if a large raw corpus is employed. Some recent approaches to Japanese PAS analysis combines neural network models with knowledge extraction from raw corpora. Shibata et al. (2016) extract selectional preferences by an unsuper-

	Predicate	NOM	ACC	DAT
(1) タクシーがNOM 客をACC 駅にDAT 送った 。 takushi-ga kyaku-wo eki-ni okutta . A taxi carried passengers to the station.	送った okutta sent/carried	タクシー takushi taxi	客 kyaku passenger	駅 eki station
(2) その 列車は 荷物をACC 運んだ 。 sono ressha-wa nimotsu-wo hakonda. The train also carried baggages.	運んだ hakonda carried	列車 ressha train	荷物 nimotsu baggage	NULL
(3) タクシーがNOM 客をACC 乗せた とき 事故にDAT 巻き込まれた 。 takushi-ga kyaku-wo noseta toki jiko-ni makikomareta. When the taxi picked up passengers, it was involved in the accident.	乗せた noseta picked up 巻き込まれた makikomareta was involved	タクシー takushi taxi タクシー takushi taxi	客 kyaku passenger NULL	NULL 事故 jiko accident
(4) この 列車に は 乗れません 。 kono ressha-ni-wa noremasen. You can not take this train.	乗れません noremasen can not take	あなた anata you	NULL	列車 ressha train

Table 1: Examples of Japanese sentences and their PAS analysis. In sentence (1), case markers (_(ga), _(wo), and _(ni)) correspond to NOM, ACC, and DAT. In example (2), the correct case marker is hidden by the topic marker _ (wa). In sentence (3), the NOM argument of the second predicate (was involved), is dropped. NULL indicates that the predicate does not have the corresponding case argument or that the case argument is not written in the sentence.

vised method that is similar to negative sampling (Mikolov et al., 2013). They then use the pre-extracted selectional preferences as one of the features to their PAS analysis model. The PAS analysis model is trained by a supervised method and the selectional preference representations are fixed during training. Using pre-trained external knowledge in the form of word embeddings has also been ubiquitous. However, such external knowledge is overwritten in the task-specific training.

The other approach to using raw corpora for PAS analysis is data augmentation. Liu et al. (2017b) generate pseudo training data from a raw corpus and use them for their zero pronoun resolution model. They generate the pseudo training data by dropping certain words or pronouns in a raw corpus and assuming them as correct antecedents. After generating the pseudo training data, they rely on ordinary supervised training based on neural networks.

In this paper, we propose a neural semi-supervised model for Japanese PAS analysis. We adopt neural adversarial training to directly exploit the advantage of using a raw corpus. Our model consists of two neural network models: a generator model of Japanese PAS analysis and a so-called "validator" model of the generator prediction. The generator neural network is a model that predicts probabilities of candidate arguments of each predicate using RNN-based features and a head-selection model (Zhang et al., 2017). The validator neural network gets inputs from the generator and scores them. This validator can score the generator prediction even when PAS gold labels are not available. We apply supervised learning to the generator and unsupervised learning to the entire network using a raw corpus.

Our contributions are summarized as follows: (1) a novel adversarial training model for PAS analysis; (2) learning from a raw corpus as a source of external knowledge; and (3) as a result, we achieve state-of-the-art performance on Japanese PAS analysis.

2 Task Description

Japanese PAS analysis determines essential case roles of words for each predicate: who did what to whom. In many languages, such as English, case roles are mainly determined by word order. However, in Japanese, word order is highly flexible. In Japanese, major case roles are the nominative case (NOM), the accusative case (ACC) and the dative case (DAT), which roughly correspond to Japanese surface case markers: _(ga), _(wo), and _(ni). These case markers are often hidden by topic markers, and case arguments are also often

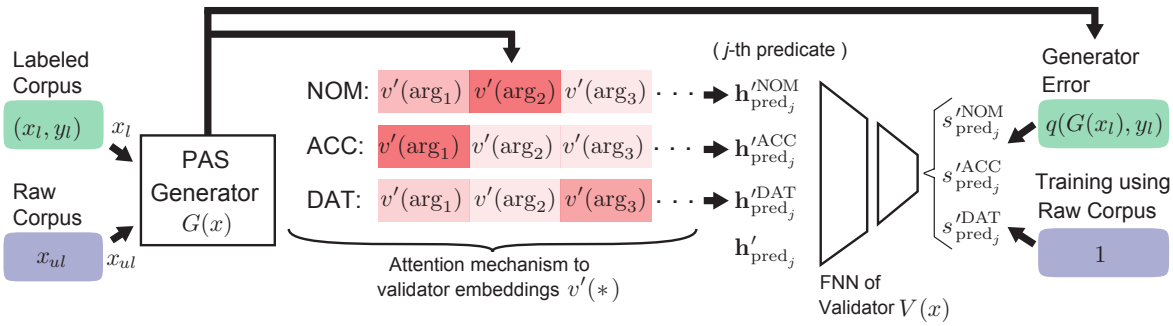


Figure 1: The overall model of adversarial training with a raw corpus. The PAS generator $G(x)$ and validator $V(x)$. The validator takes inputs from the generator as a form of the attention mechanism. The validator itself is a simple feed-forward network with inputs of j-th predicate and its argument representations: $\{h'_{\mathrm{pred}_j}, h'^{\mathrm{case}_k}_{\mathrm{pred}_j}\}$. The validator returns scores for three cases and they are used for both the supervised training of the validator and the unsupervised training of the generator. The supervised training of the generator is not included in this figure.

omitted.

We explain two detailed tasks of PAS analysis: case analysis and zero anaphora resolution. In Table 1, we show four example Japanese sentences and their PAS labels. PAS labels are attached to nominative, accusative and dative cases of each predicate. Sentence (1) has surface case markers that correspond to argument cases.

Sentence (2) is an example sentence for case analysis. Case analysis is a task to find hidden case markers of arguments that have direct dependencies to their predicates. Sentence (2) does not have the nominative case marker _(ga). It is hidden by the topic case marker _(wa). Therefore, a case analysis model has to find the correct NOM case argument (train).

Sentence (3) is an example sentence for zero anaphora resolution. Zero anaphora resolution is a task to find arguments that do not have direct dependencies to their predicates. At the second predicate " "(was involved), the correct nominative argument is " "(taxi), while this does not have direct dependencies to the second predicate. A zero anaphora resolution model has to find " "(taxi) from the sentence, and assign it to the NOM case of the second predicate.

In the zero anaphora resolution task, some correct arguments are not specified in the article. This is called as exophora. We consider "author" and "reader" arguments as exophora (Hangyo et al., 2013). They are frequently dropped from Japanese natural sentences. Sentence (4) is an example of dropped nominative arguments. In this sentence, the nominative argument is " " (you), but " " (you) does not appear in the sentence. This is also included in zero anaphora resolution. Except these special arguments of exophora, we focus on intra-sentential anaphora resolution in the same way as (Shibata et al., 2016; Iida et al., 2016; Ouchi et al., 2017; Matsubayashi and Inui, 2017). We also attach NULL labels to cases that predicates do not have.

3 Model

3.1 Generative Adversarial Networks

Generative adversarial networks are originally proposed in image generation tasks (Goodfellow et al., 2014; Salimans et al., 2016; Springenberg, 2015). In the original model in Goodfellow et al. (2014), they propose a generator G and a discriminator D. The discriminator D is trained to devide the real data distribution $p_{data}(\mathbf{x})$ and images generated from the noise samples $\mathbf{z}^{(i)} \in \mathcal{D}_{\mathbf{z}}$ from noise prior $p(\mathbf{z})$. The discriminator loss is

$$\mathcal{L}_D = -\big(\mathbb{E}_{\mathbf{x}\sim p_{data}(\mathbf{x})}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p_z(\mathbf{z})}[\log(1 - D(G(\mathbf{z})))]\big) \quad , \qquad (1)$$

and they train the discriminator by minimizing this loss while fixing the generator G. Similarly, the generator G is trained through minimizing

$$\mathcal{L}_G = \frac{1}{|\mathcal{D}_{\mathbf{z}}|}\sum_i \Big[\log\Big(1 - D(G(\mathbf{z}^{(i)}))\Big)\Big] \quad , \qquad (2)$$

while fixing the discriminator D. By doing this, the discriminator tries to descriminate the generated images from real images, while the generator tries to generate images that can deceive the adversarial discriminator. This training scheme is applied for many generative tasks including sentence generation (Subramanian et al., 2017), machine translation (Britz et al., 2017), dialog generation (Li et al., 2017), and text classification (Liu et al., 2017a).

3.2 Proposed Adversarial Training Using Raw Corpus

Japanese PAS analysis and many other syntactic analyses in NLP are not purely generative, and we can make use of a raw corpus instead of the numerical noise distribution $p(\mathbf{z})$. In this work, we use an adversarial training method using a raw corpus, combined with ordinary supervised learning using an annotated corpus. Let $\mathbf{x}_l \in \mathcal{D}_l$ indicate labeled data and $p(\mathbf{x}_l)$ indicate their label distribution. We also use unlabeled data $\mathbf{x}_{ul} \in \mathcal{D}_{ul}$ later. Our generator G can be trained by the cross entropy loss with labeled data:

$$\mathcal{L}_{G/SL} = -\mathbb{E}_{\mathbf{x}_l, y \sim p(\mathbf{x}_l)} \big[\log G(\mathbf{x}_l)\big] \quad . \tag{3}$$

Supervised training of the generator works by minimizing this loss. Note that we follow the notations of Subramanian et al. (2017) in this subsection.

In addition, we train a so-called validator against the generator errors. We use the term "validator" instead of "discriminator" for our adversarial training. Unlike the discriminator that is used for dividing generated images and real images, our validator is used to score the generator results. Assume that $\mathbf{y}_l$ is the true labels and $G(\mathbf{x}_l)$ is the predicted label distribution of data $\mathbf{x}_l$ from the generator. We define the labels of the generator errors as:

$$q(G(\mathbf{x}_l), \mathbf{y}_l) = \delta_{\arg\max[G(\mathbf{x}_l)],\ \mathbf{y}_l} \quad , \tag{4}$$

where $\delta_{i,j} = 1$ only if $i = j$, otherwise $\delta_{i,j} = 0$. This means that q is equal to 1 if the argument that the generator predicts is correct, otherwise 0. We use this generator error for training labels of the following validator. The inputs of the validator are both the generator outputs $G(\mathbf{x})$ and data $\mathbf{x} \in \mathcal{D}$. The validator can be written as $V(G(\mathbf{x}))$. The validator V is trained with labeled data $\mathbf{x}_l$ by

$$\mathcal{L}_{V/SL} = -\mathbb{E}_{\mathbf{x}_l, y \sim q(G(\mathbf{x}_l), \mathbf{y}_l)} \big[\log V(G(\mathbf{x}_l))\big] \quad , \tag{5}$$

while fixing the generator G. This equation means that the validator is trained with labels of the generator error $q(G(\mathbf{x}_l), \mathbf{y}_l)$.

Once the validator is trained, we train the generator with an unsupervised method. The generator G is trained with unlabeled data $\mathbf{x}_{ul} \in \mathcal{D}_{ul}$ by minimizing the loss

$$\mathcal{L}_{G/UL} = -\frac{1}{|\mathcal{D}_{ul}|} \sum_i \big[\log V(G(\mathbf{x}_{ul}^{(i)}))\big] \quad , \tag{6}$$

while fixing the validator V. This generator training loss using the validator can be explained as follows. The generator tries to increase the validator scores to 1, while the validator is fixed. If the validator is well-trained, it returns scores close to 1 for correct PAS labels that the generator outputs, and 0 for wrong labels. Therefore, in Equation (6), the generator tries to predict correct labels in order to increase the scores of fixed validator. Note that the validator has a sigmoid function for the output of scores. Therefore output scores of the validator are in $[0, 1]$.

We first conduct the supervised training of generator network with Equation (3). After this, following Goodfellow et al. (2014), we use k-steps of the validator training and one-step of the generator training. We also alternately conduct l-steps of supervised training of the generator. The entire loss function of this adversarial training is

$$\mathcal{L} = \mathcal{L}_{G/SL} + \mathcal{L}_{V/SL} + \mathcal{L}_{G/UL} \quad . \tag{7}$$

Our contribution is that we propose the validator and train it against the generator errors, instead of discriminating generated data from real data. Salimans et al. (2016) explore the semi-supervised learning using adversarial training for K-classes image classification tasks. They add a new class of images that are generated by the generator and classify them. Miyato et al. (2016) propose virtual adversarial training for semi-supervised learning. They exploit unlabeled data for continuous smoothing of data distributions based on the

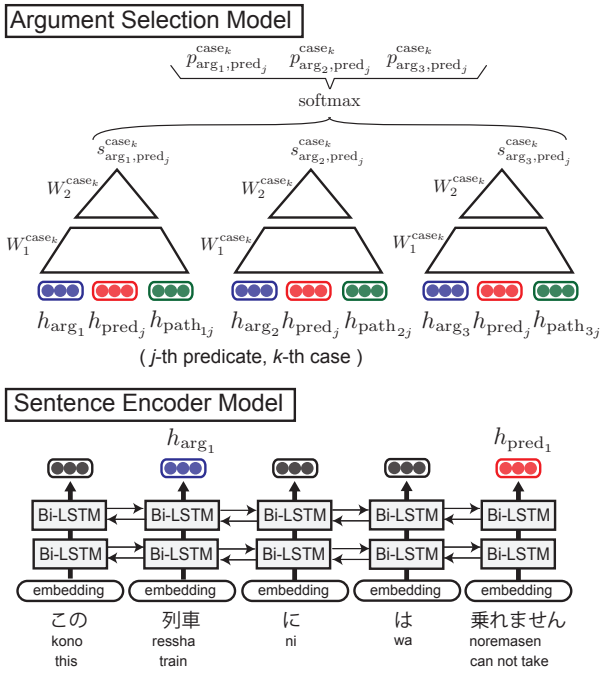


Figure 2: The generator of PAS. The sentence encoder is a three-layer bi-LSTM to compute the distributed representations of a predicate and its arguments: h_{pred_i} and h_{arg_i}. The argument selection model is two-layer feedforward neural networks to compute the scores, $s^{\text{case}_k}_{\text{arg}_i,\text{pred}_j}$, of candidate arguments for each case of a predicate.

adversarial perturbation of Goodfellow et al. (2015). These studies, however, do not use the counterpart neural networks for learning structures of unlabeled data.

In our Japanese PAS analysis model, the generator corresponds to the head-selection-based neural network for Japanese anaphora resolution. Figure 1 shows the entire model. The labeled data correspond to the annotated corpora and the labels correspond to the PAS argument labels. The unlabeled data correspond to raw corpora. We explain the details of the generator and the validator neural networks in Sec.3.3 and Sec.3.4 in turn.

3.3 Generator of PAS Analysis

The generator predicts the probabilities of arguments for each of the NOM, ACC and DAT cases of a predicate. As shown in Figure 2, the generator consists of a sentence encoder and an argument selection model. In the sentence encoder, we use a three-layer bidirectional-LSTM (bi-LSTM) to read the whole sentence and extract both global and local features as distributed representations. The argument selection model consists of a two-layer feedforward neural network (FNN) and a softmax function.

For the sentence encoder, inputs are given as a sequence of embeddings $v(x)$, each of which consist of word x, its inflection from, POS and detailed POS. They are concatenated and fed into the bi-LSTM layers. The bi-LSTM layers read these embeddings in forward and backward order and outputs the distributed representations of a predicate and a candidate argument: h_{pred_j} and h_{arg_i}. Note that we also use the exophora entities, i.e., an author and a reader, as argument candidates. Therefore, we use specific embeddings for them. These embeddings are not generated by the bi-LSTM layers but are directly used in the argument selection model.

We also use path embeddings to capture a dependency relation between a predicate and its candidate argument as used in Roth and Lapata (2016). Although Roth and Lapata (2016) use a one-way LSTM layer to represent the dependency path from a predicate to its potential argument, we use a bi-LSTM layer for this purpose. We feed the embeddings of words and POS tags to the bi-LSTM layer. In this way, the resulting path embedding represents both predicate-to-argument and argument-to-predicate paths. We concatenate the bidirectional path embeddings to generate $h_{\text{path}_{ij}}$, which represents the dependency relation between the predicate j and its candidate argument i.

For the argument selection model, we apply the argument selection model (Zhang et al., 2017) to evaluate the relation between a predicate and its potential argument for each argument case. In the argument selection model, a single FNN is repeatedly used to calculate scores for a child word and its head candidate word, and then a softmax function calculates normalized probabilities of candidate heads. We use three different FNNs that correspond to the NOM, ACC and DAT cases. These three FNNs have the same inputs of the distributed representations of j-th predicate h_{pred_j}, i-th candidate argument h_{arg_i} and path

embedding $h_{\text{path}_{ij}}$ between the predicate j and candidate argument i. The FNNs for NOM, ACC and DAT compute the argument scores $s^{\text{case}_k}_{\text{arg}_i,\text{pred}_j}$, where $\text{case}_k \in \{\text{NOM}, \text{ACC}, \text{DAT}\}$. Finally, the softmax function computes the probability $p(\text{arg}_i|\text{pred}_j,\text{case}_k)$ of candidate argument i for case k of j-th predicate as:

$$p(\text{arg}_i|\text{pred}_j,\text{case}_k) = \frac{\exp\left(s^{\text{case}_k}_{\text{arg}_i,\text{pred}_j}\right)}{\sum_{\text{arg}_i} \exp\left(s^{\text{case}_k}_{\text{arg}_i,\text{pred}_j}\right)}. \quad (8)$$

Our argument selection model is similar to the neural network structure of Matsubayashi and Inui (2017). However, Matsubayashi and Inui (2017) does not use RNNs to read the whole sentence. Their model is also designed to choose a case label for a pair of a predicate and its argument candidate. In other words, their model can assign the same case label to multiple arguments by itself, while our model does not. Since case arguments are almost unique for each case of a predicate in Japanese, Matsubayashi and Inui (2017) select the argument that has the highest probability for each case, even though probabilities of case arguments are not normalized over argument candidates. The model of Ouchi et al. (2017) has the same problem.

3.4 Validator

We exploit a validator to train the generator using a raw corpus. It consists of a two-layer FNN to which embeddings of a predicate and its arguments are fed. For predicate j, the input of the FNN is the representations of the predicate h'_{pred_j} and three arguments $\left\{h'^{\text{ NOM}}_{\text{pred}_j}, h'^{\text{ ACC}}_{\text{pred}_j}, h'^{\text{ DAT}}_{\text{pred}_j}\right\}$ that are inferred by the generator. The two-layer FNN outputs three values, and then three sigmoid functions compute the scores of scalar values in a range of $[0, 1]$ for the NOM, ACC and DAT cases: $\left\{s'^{\text{ NOM}}_{\text{pred}_j}, s'^{\text{ ACC}}_{\text{pred}_j}, s'^{\text{ DAT}}_{\text{pred}_j}\right\}$. These scores are the outputs of the validator $D(x)$. We use dropout of 0.5 at the FNN input and hidden layer.

The generator and validator networks are coupled by the attention mechanism, or the weighted sum of the validator embeddings. As shown in Equation (8), we compute a probability distribution of candidate arguments. We use the weighted sum of embeddings $v'(x)$ of candidate arguments to compute the input representations of the validator:

$$\begin{aligned} h'^{\text{ case}_k}_{\text{pred}_j} &= E_{\mathbf{x}\sim p(\text{arg}_i)}[v'(\mathbf{x})] \\ &= \sum_{\text{arg}_i} p(\text{arg}_i|\text{pred}_j,\text{case}_k) v'(\text{arg}_i). \end{aligned}$$

This summation is taken over candidate arguments in the sentence and the exophora entities. Note that we use embeddings $v'(x)$ for the validator that are different from the embeddings $v(x)$ for the generator, in order to separate the computation graphs of the generator and the validator neural networks except the joint part. We use this weighted sum by the softmax outputs instead of the argmax function. This allows the backpropagation through this joint. We also feed the embedding of a predicate to the validator:

$$h'_{\text{pred}_j} = v'(\text{pred}_j). \quad (9)$$

Note that the validator is a simple neural network compared with the generator. The validator has limited inputs of predicates and arguments and no inputs of other words in sentences. This allows the generator to overwhelm the validator during the adversarial training.

3.5 Implementation Details

The neural networks are trained using backpropagation. The backpropagation has been done to the word and POS tags. We use Adam (Kingma and Ba, 2015) at the initial training of the generator network for the gradient learning rule. In adversarial learning, Adagrad (Duchi et al., 2010) is suitable because of the stability of learning. We use pre-trained word embeddings from 100M sentences from Japanese web corpus by word2vec (Mikolov et al., 2013). Other embeddings and hidden weights of neural networks are randomly initialized.

For adversarial training, we first train the generator for two epochs by the supervised method, and train the validator while fixing the generator for another epoch. This is because the validator training preceding the generator training makes the validator result worse. After this, we alternately do the unsupervised training of the generator ($L_{G/UL}$),

Type	Value
Size of hidden layers of FNNs	1,000
Size of Bi-LSTMs	256
Dim. of word embedding	100
Dim. of POS, detailed POS, inflection form tags	10, 10, 9
Minibatch size for the generator and validator	16, 1

Table 2: Parameters for neural network structure and training.

KWDLC	# snt	# of dep	# of zero
Train	11,558	9,227	8,216
Dev.	1,585	1,253	821
Test	2,195	1,780	1,669

Table 3: KWDLC data statistics.

k-times of supervised training of the validator ($L_{V/SL}$) and l-times of supervised training of the generator ($L_{G/SL}$).

We use the $N(L_{G/UL})/N(L_{G/SL}) = 1/4$ and $N(L_{V/SL})/N(L_{G/SL}) = 1/4$, where $N(\cdot)$ indicates the number of sentences used for training. Also we use minibatch of 16 sentences for both supervised and unsupervised training of the generator, while we do not use minibatch for validator training. Therefore, we use $k = 16$ and $l = 4$. Other parameters are summarized in Table 2.

4 Experiments

4.1 Experimental Settings

Following Shibata et al. (2016), we use the KWDLC (Kyoto University Web Document Leads Corpus) corpus (Hangyo et al., 2012) for our experiments.[1] This corpus contains various Web documents, such as news articles, personal blogs, and commerce sites. In KWDLC, lead three sentences of each document are annotated with PAS structures including zero pronouns. For a raw corpus, we use a Japanese web corpus created by Hangyo et al. (2012), which has no duplicated sentences with KWDLC. This raw corpus is automatically parsed by the Japanese dependency parser KNP.

We focus on intra-sentential anaphora resolution, and so we apply a preprocess to KWDLC. We regard the anaphors whose antecedents are in the preceding sentences as NULL in the same way as Ouchi et al. (2015); Shibata et al. (2016). Tables 3 and 4 list the statistics of KWDLC.

We use the exophora entities, i.e., an author and a reader, following the annotations in KWDLC. We also assign author/reader labels to the following expressions in the same way as Hangyo et al. (2013); Shibata et al. (2016):

author “ ” (I), “ ” (I), “ ” (we), “ ” (our company)

reader “ ” (you), “ ” (you), “ ” (customer), “ ” (you all)

Following Ouchi et al. (2015) and Shibata et al. (2016), we conduct two kinds of analysis: (1) case analysis and (2) zero anaphora resolution. Case analysis is the task to determine the correct case labels when predicates and their arguments have direct dependencies but their case markers are hidden by surface markers, such as topic markers. Zero anaphora resolution is a task to find certain case arguments that do not have direct dependencies to their predicates in the sentence.

Following Shibata et al. (2016), we exclude predicates that the same arguments are filled in multiple cases of a predicate. This is relatively uncommon and 1.5 % of the whole cor-

KWDLC	NOM	ACC	DAT
# of dep	7,224	1,555	448
# of zero	6,453	515	1,248

Table 4: KWDLC training data statistics for each case.

	Case	Zero
Ouchi+ 2015	76.5	42.1
Shibata+ 2016	89.3	53.4
Gen	91.5	56.2
Gen+Adv	92.0‡	58.4‡

Table 5: The results of case analysis (Case) and zero anaphora resolution (Zero). We use F-measure as an evaluation measure. ‡ denotes that the improvement is statistically significant at $p < 0.05$, compared with Gen using paired t-test.

[1] The KWDLC corpus is available at http://nlp.ist.i.kyoto-u.ac.jp/EN/index.php?KWDLC

	Case analysis			Zero anaphora resolution		
Model	NOM	ACC	DAT	NOM	ACC	DAT
Ouchi+ 2015	87.4	40.2	27.6	48.8	0.0	10.7
Shibata+ 2016	94.1	75.6	30.0	57.7	17.3	37.8
Gen	95.3	83.6	39.7	60.7	30.4	41.2
Gen+Adv	95.3	85.4	51.5	62.3	31.1	44.6

Table 6: The detailed results of case analysis and zero anaphora resolution for the NOM, ACC and DAT cases. Our models outperform the existing models in all cases. All values are evaluated with F-measure.

	Case	Zero
Gen	91.5	56.2
Gen+Aug	91.2	57.0
Gen+Adv	$92.0^{\ddagger}$	$58.4^{\ddagger}$

Table 7: The comparisons of Gen+Adv with Gen and the data augmentation model (Gen+Aug). ‡ denotes that the improvement is statistically significant at $p < 0.05$, compared with Gen+Aug.

pus are excluded. Predicates are marked in the gold dependency parses. Candidate arguments are just other tokens than predicates. This setting is also the same as Shibata et al. (2016).

All performances are evaluated with micro-averaged F-measure (Shibata et al., 2016).

4.2 Experimental Results

We compare two models: the supervised generator model (Gen) and the proposed semi-supervised model with adversarial training (Gen+Adv). We also compare our models with two previous models: Ouchi et al. (2015) and Shibata et al. (2016), whose performance on the KWDLC corpus is reported.

Table 5 lists the experimental results. Our models (Gen and Gen+Adv) outperformed the previous models. Furthermore, the proposed model with adversarial training (Gen+Adv) was significantly better than the supervised model (Gen).

4.3 Comparison with Data Augmentation Model

We also compare our GAN-based approach with data augmentation techniques. A data augmentation approach is used in Liu et al. (2017b). They automatically process raw corpora and make drops of words with some rules. However, it is difficult to directly apply their approach to Japanese PAS analysis because Japanese zero-pronoun depends on dependency trees. If we make some drops of arguments of predicates in sentences, this can cause lacks of nodes in dependency trees. If we prune some branches of dependency trees of the sentence, this cause the data bias problem.

Therefore we use existing training corpora and word embeddings for the data augmentation. First we randomly choose an argument word w in the training corpus and then swap it with another word w' with the probability of $p(w, w')$. We choose top-20 nearest words to the original word w in the pre-trained word embedding as candidates of swapped words. The probability is defined as $p(w, w') \propto [v(w)^\top v(w')]^r$, where $r = 10$. This probability is normalized by top-20 nearest words. We then merge this pseudo data and the original training corpus and train the model in the same way with the Gen model. We conducted several experiments and found that the model trained with the same amount of the pseudo data as the training corpus achieved the best result.

Table 7 shows the results of the data augmentation model and the GAN-based model. Our Gen+Adv model performs better than the data augmented model. Note that our data augmentation model does not use raw corpora directly.

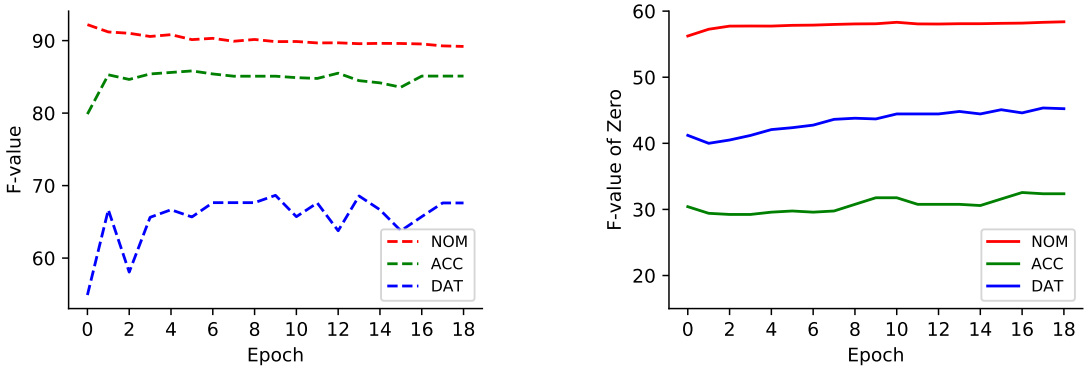


Figure 3: Left: validator scores with the development set during adversarial training epochs. Right: generator scores for Zero with the development set during adversarial training epochs.

4.4 Discussion

4.4.1 Result Analysis

We report the detailed performance for each case in Table 6. Among the three cases, zero anaphora resolution of the ACC and DAT cases is notoriously difficult. This is attributed to the fact that these ACC and DAT cases are fewer than the NOM case in the corpus as shown in Table 4. However, we can see that our proposed model, Gen+Adv, performs much better than the previous models especially for the ACC and DAT cases. Although the number of training instances of ACC and DAT is much smaller than that of NOM, our semi-supervised model can learn PAS for all three cases using a raw corpus. This indicates that our model can work well in resource-poor cases.

We analyzed the results of Gen+Adv by comparing with Gen and the model of Shibata et al. (2016). Here, we focus on the ACC and DAT cases because their improvements are notable.

- “ ”
  It is bothersome to wash, classify and recycle spent packs.

In this sentence, the predicates “ ” (wash), “ ” (classify), “( ) ” (recycle) takes the same ACC argument, “ ” (pack). This is not so easy for Japanese PAS analysis because the actual ACC case marker “ ” (wo) of “ ” (pack) is hidden by the topic marker “ ” (wa). The Gen+Adv model can detect the correct argument while the model of Shibata et al. (2016) fails. In the Gen+Adv model, each predicate gives a high probability to “ ” (pack) as an ACC argument and finally chooses this. We found many examples similar to this and speculate that our model captures a kind of selectional preferences.

The next example is an error of the DAT case by the Gen+Adv model.

- “ ”
  please leave every professional field (to ϕ)

The gold label of this DAT case (to ϕ) is NULL because this argument is not written in the sentence. However, the Gen+Adv model judged the DAT argument as “author”. Although we cannot specify ϕ as “author” only from this sentence, “author” is a possible argument depending on the context.

4.4.2 Validator Analysis

We also evaluate the performance of the validator during the adversarial training with raw corpora. Figure 3 shows the validator performance and the generator performance of Zero on the development set. The validator score is evaluated with the outputs of generator.

We notice that the NOM case and the other two cases have different curves in both graphs. This can be explained by the speciality of the NOM case. The NOM case has much more author/reader expressions than the other cases. The prediction of author/reader expressions depends not only on selectional preferences of predicates and arguments but on the whole of sentences. Therefore the validator that relies only on predicate and argument representations cannot predict author/reader expressions well.

In the ACC and DAT cases, the scores of the generator and validator increase in the first epochs. This suggests that the validator learns the weakness of the generator and vice versa. However, in later epochs, the scores of the generator increase with fluctuation, while the scores of the validator saturates. This suggests that the generator gradually becomes stronger than the validator.

5 Related Work

Shibata et al. (2016) proposed a neural network-based PAS analysis model using local and global features. This model is based on the non-neural model of Ouchi et al. (2015). They achieved state-of-the-art results on case analysis and zero anaphora resolution using the KWDLC corpus. They use an external resource to extract selectional preferences. Since our model uses an external resource, we compare our model with the models of Shibata et al. (2016) and Ouchi et al. (2015).

Ouchi et al. (2017) proposed a semantic role labeling-based PAS analysis model using Grid-RNNs. Matsubayashi and Inui (2017) proposed a case label selection model with feature-based neural networks. They conducted their experiments on NAIST Text Corpus (NTC) (Iida et al., 2007, 2016). NTC consists of newspaper articles, and does not include the annotations of author/reader expressions that are common in Japanese natural sentences.

6 Conclusion

We proposed a novel Japanese PAS analysis model that exploits a semi-supervised adversarial training. The generator neural network learns Japanese PAS and selectional preferences, while the validator is trained against the generator errors. This validator enables the generator to be trained from raw corpora and enhance it with external knowledge. In the future, we will apply this semi-supervised training method to other NLP tasks.

Acknowledgment

This work was supported by JST CREST Grant Number JPMJCR1301, Japan and JST ACT-I Grant Number JPMJPR17U8, Japan.

References

Denny Britz, Quoc Le, and Reid Pryzant. 2017. Effective domain mixing for neural machine translation. In Proceedings of the Second Conference on Machine Translation. Association for Computational Linguistics, Copenhagen, Denmark, pages 118–126. http://www.aclweb.org/anthology/W17-4712.

John Duchi, Elad Hazan, and Yoram Singer. 2010. Adaptive subgradient methods for online learning and stochastic optimization. UCB/EECS-2010-24.

Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron C. Courville, and Yoshua Bengio. 2014. Generative adversarial nets. In Advances in Neural Information Processing Systems 27: Annual Conference on Neural Information Processing Systems 2014, December 8-13 2014, Montreal, Quebec, Canada. pages 2672–2680.

Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. 2015. Explaining and harnessing adversarial examples. In Proceedings of the International Conference on Learning Representations (ICLR).

Masatsugu Hangyo, Daisuke Kawahara, and Sadao Kurohashi. 2012. Building a diverse document leads corpus annotated with semantic relations. In Proceedings of the 26th Pacific Asia Conference on Language, Information, and Computation. Faculty of Computer Science, Universitas Indonesia, Bali,Indonesia, pages 535–544.

Masatsugu Hangyo, Daisuke Kawahara, and Sadao Kurohashi. 2013. Japanese zero reference resolution considering exophora and author/reader mentions. In Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics, Seattle, Washington, USA, pages 924–934.

Ryu Iida, Mamoru Komachi, Kentaro Inui, and Yuji Matsumoto. 2007. Annotating a japanese text corpus with predicate-argument and coreference relations. In Proceedings of the Linguistic Annotation Workshop, LAW@ACL 2007, Prague, Czech Republic, June 28-29, 2007. pages 132–139.

Ryu Iida, Kentaro Torisawa, Jong-Hoon Oh, Canasai Kruengkrai, and Julien Kloetzer. 2016. Intra-sentential subject zero anaphora resolution using multi-column convolutional neural network. In Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics, Austin, Texas, pages 1244–1254. https://aclweb.org/anthology/D16-1132.

D. P. Kingma and J. Ba. 2015. Adam: A method for stochastic optimization. In Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers).

Jiwei Li, Will Monroe, Tianlin Shi, Sébastien Jean, Alan Ritter, and Dan Jurafsky. 2017. Adversarial learning for neural dialogue generation. In Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics, Copenhagen, Denmark, pages 2157–2169.

Pengfei Liu, Xipeng Qiu, and Xuanjing Huang. 2017a. Adversarial multi-task learning for text classification. In Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics, Vancouver, Canada, pages 1–10.

Ting Liu, Yiming Cui, Qingyu Yin, Wei-Nan Zhang, Shijin Wang, and Guoping Hu. 2017b. Generating and exploiting large-scale pseudo training data for zero pronoun resolution. In Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics, Vancouver, Canada, pages 102–111.

Yuichiroh Matsubayashi and Kentaro Inui. 2017. Revisiting the design issues of local models for japanese predicate-argument structure analysis. In Proceedings of the Eighth International Joint Conference on Natural Language Processing (Volume 2: Short Papers). Asian Federation of Natural Language Processing, Taipei, Taiwan, pages 128–133.

Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient estimation of word representations in vector space. volume abs/1301.3781. http://arxiv.org/abs/1301.3781.

Takeru Miyato, Shin-ichi Maeda, Masanori Koyama, Ken Nakae, and Shin Ishii. 2016. Distributional smoothing by virtual adversarial examples. In Proceedings of the International Conference on Learning Representations (ICLR).

Hiroki Ouchi, Hiroyuki Shindo, Kevin Duh, and Yuji Matsumoto. 2015. Joint case argument identification for japanese predicate argument structure analysis. In Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers). Association for Computational Linguistics, Beijing, China, pages 961–970.

Hiroki Ouchi, Hiroyuki Shindo, and Yuji Matsumoto. 2017. Neural modeling of multi-predicate interactions for japanese predicate argument structure analysis. In Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics, Vancouver, Canada, pages 1591–1600.

Michael Roth and Mirella Lapata. 2016. Neural semantic role labeling with dependency path embeddings. In Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics, Berlin, Germany, pages 1192–1202.

Tim Salimans, Ian Goodfellow, Wojciech Zaremba, Vicki Cheung, Alec Radford, Xi Chen, and Xi Chen. 2016. Improved techniques for training gans. In D. D. Lee, M. Sugiyama, U. V. Luxburg, I. Guyon, and R. Garnett, editors, Advances in Neural Information Processing Systems 29. Curran Associates, Inc., pages 2234–2242.

Ryohei Sasano and Sadao Kurohashi. 2011. A discriminative approach to japanese zero anaphora resolution with large-scale lexicalized case frames. In Proceedings of 5th International Joint Conference on Natural Language Processing. Asian Federation of Natural Language Processing, Chiang Mai, Thailand, pages 758–766.

Tomohide Shibata, Daisuke Kawahara, and Sadao Kurohashi. 2016. Neural network-based model for japanese predicate argument structure analysis. In Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics, pages 1235–1244.

Jost Tobias Springenberg. 2015. Unsupervised and semi-supervised learning with categorical generative adversarial networks.

Sandeep Subramanian, Sai Rajeswar, Francis Dutil, Chris Pal, and Aaron Courville. 2017. Adversarial generation of natural language. In Proceedings of the 2nd Workshop on Representation Learning for NLP. Association for Computational Linguistics, Vancouver, Canada, pages 241–251.

Xingxing Zhang, Jianpeng Cheng, and Mirella Lapata. 2017. Dependency parsing as head selection. In Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 1, Long Papers. Association for Computational Linguistics, Valencia, Spain, pages 665–676.